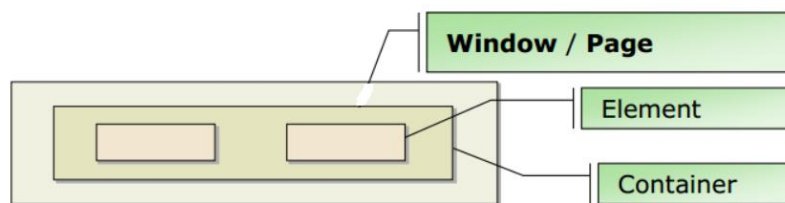


Layout

Arrangement

```
<Window ...>
  <StackPanel>
    <Button>bt1</Button>
    <Button>bt2</Button>
  </StackPanel>
  ...
</Window>
```



Der wichtigste Aspekt beim Layouting eines WPF-Fensters ist das Arrangement der verschiedenen UI-Elemente. Dabei spielen insbesondere die verschiedenen Layoutcontainer und deren spezielle Verhalten sowie die Art, wie diese ineinander verschachtelt sind, eine große Rolle.

WPF ist grundsätzlich sehr modular strukturiert, was dazu führt, dass beliebige Elemente auf sehr individuelle Weise ineinander gelegt werden können. Jedes Element, welches die ‚Content‘-Eigenschaft besitzt, kann jedes beliebige andere Element fassen, also sowohl andere Control-Objekte, als auch andere Layoutcontainer, die wiederum über die ‚Children‘-Eigenschaft weitere Elemente fassen können.

Jeder Layoutcontainer bringt dabei seine eigene Logik mit, wie er beinhaltende Elemente strukturiert. Dabei definiert er für jedes Element eine Standard-Ausdehnung und –Position, welche je nach Layoutcontainer individuell angepasst werden können.

Das ‚Feintuning‘ erfolgt dann über Layouteigenschaften der einzelnen Elemente, wie z.B. Height, Width, Horizontal- und VerticalAlignment, sowie Margin und Padding.

DIP – device-independent pixels

- Absolute Größen (Abstände, Längen, Positionen) werden in DIP (device-independent pixels) angegeben
- Keine ‚echten‘ Pixel, sondern abstrakte Einheit basierend auf realen Größen
- Pixelanzahl eines DIP berechnet sich individuell anhand von Bildschirmgröße und –auflösung
 - Bei Standardbildschirmen (96 dots per inch): 1 DIP = 1 Pixel -> ca. 38 DIP/cm

-> Layout behält auf allen Bildschirmen sein Verhältnis bei und ‚sieht gleich aus‘

DIP (Device Independent Pixel) ist eine Maßeinheit in WPF, die verwendet wird, um die Größe und Position von Elementen auf dem Bildschirm zu definieren. Im Gegensatz zu Pixeln, die von der tatsächlichen Auflösung des Bildschirms abhängen, sind DIPs eine abstrakte Einheit, die auf einer logischen Auflösung basieren. Standardmäßig entspricht 1 DIP 1/96 Zoll, was einer logischen Auflösung von 96 DPI (dots per inch) entspricht.

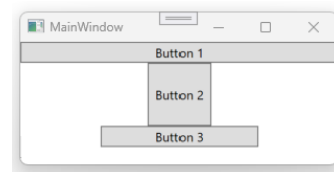
Die Verwendung von DIPs bietet den Vorteil, dass die Benutzeroberfläche in WPF unabhängig von der tatsächlichen Bildschirmauflösung konsistent und skalierbar bleibt. Wenn eine Anwendung auf einem Bildschirm mit höherer Auflösung angezeigt wird, werden die Elemente automatisch skaliert, um ihre korrekte Darstellung beizubehalten. Dadurch wird sichergestellt, dass die Elemente weder zu klein noch zu groß erscheinen.

DIPs können direkt als Wert angegeben werden. Unter bestimmten Voraussetzungen (innerhalb bestimmter Layoutcontainer) können relative Einheiten wie "Auto" oder "*" verwendet werden, um die Größe dynamisch anzupassen. Wird kein expliziter Wert angegeben, wird das definierte Standardverhalten des umgebenden Elements verwendet.

Height und Width

- Height: Höhe eines Elements
- Width: Breite eines Elements
- Werden Height/Width nicht explizit gesetzt, ergibt sich die Ausdehnung eines Elements durch das Standardverhalten des umgebenden Elements
- MaxHeight / MinHeight: Maximale/Minimale Höhe eines Elements
- MaxWidth / MinWidth: Maximale/Minimale Breite eines Elements

```
<StackPanel>  
    <Button>Button 1</Button>  
    <Button Height="60" Width="60">Button 2</Button>  
    <Button MaxWidth="150">Button 3</Button>  
</StackPanel>
```



Es gibt verschiedene Eigenschaften zur Steuerung der Größe von Elementen:

Die Height-Eigenschaft definiert die Höhe eines Elements, während die Width-Eigenschaft die Breite festlegt. Es können feste Werte angegeben oder relative Einheiten wie "Auto" verwenden, um die Größe automatisch anzupassen, wobei bei letzterem der jeweilige Layoutcontainer dies erlauben muss. Wird kein expliziter Wert angegeben, wird das definierte Standardverhalten des umgebenden Elements verwendet.

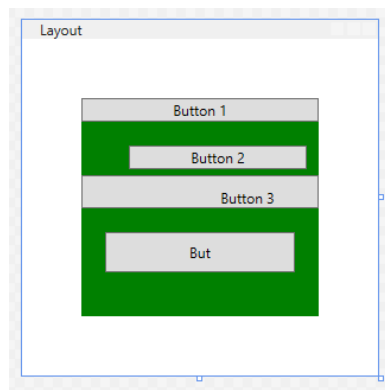
Die MaxHeight- und MaxWidth-Eigenschaften legen die maximale Größe eines Elements fest. Wenn der Inhalt oder andere Faktoren zu einer größeren Größe führen würden, wird das Element automatisch skaliert oder beschnitten, um innerhalb dieser Grenzen zu bleiben.

Die MinHeight- und MinWidth-Eigenschaften definieren die minimale Größe eines Elements. Falls der Inhalt oder andere Faktoren eine kleinere Größe verursachen würden, wird das Element automatisch skaliert oder mit Leerzeichen aufgefüllt, um die Mindestgröße zu erreichen.

Margin und Padding

- Margin
 - Abstand vom Element nach außen (zu Container)
- Padding
 - Abstand vom Element nach innen (zu Content)

```
<StackPanel Margin="50" Background="Green">
    <Button>Button 1</Button>
    <Button Margin="40, 20, 10, 5">Button 2</Button>
    <Button Padding="80, 10, 0, 0">Button 3</Button>
    <Button Margin="20" Padding="70, 8">Button 4</Button>
</StackPanel>
```



Die Eigenschaften Margin und Padding werden verwendet, um den Leerraum um ein Element herum zu definieren und das Layout zu steuern.

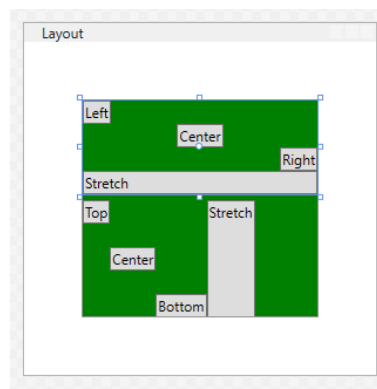
1. Margin: Die Margin-Eigenschaft wird verwendet, um den äußeren Abstand eines Elements zu seiner Umgebung zu definieren. Der Wert kann positive oder negative Werte haben.
2. Padding: Die Padding-Eigenschaft definiert den Innenabstand eines Elements, also den Abstand zwischen dem Inhalt des Elements und dem Rand des Elements selbst. Auch hier kann der Wert positiv oder negativ sein.

Es ist zu beachten, dass die Auswirkungen beider Eigenschaften durch festgelegte Ausdehnungen (per Height und Width) beeinträchtigt werden können. Zudem sind die Eigenschaften additiv, d.h. bei Festlegung einer Padding in einem äußeren und einer Margin bei einem inneren Element addieren sich die beiden Abstände auf, was zu unerwünschten Ergebnissen führen kann.

Alignment

- **HorizontalAlignment / VerticalAlignment**
 - Ausrichtung des Elements auf der Achse innerhalb des umgebenden Elements
 - Umgebendes Element muss dies zulassen
- **HorizontalContentAlignment / VerticalContentAlignment**
 - Ausrichtung des Contents auf der Achse innerhalb des Elements

```
<StackPanel Margin="50" Background="Green">
  <StackPanel Orientation="Vertical">
    <Button HorizontalAlignment="Left">Left</Button>
    <Button HorizontalAlignment="Center">Center</Button>
    <Button HorizontalAlignment="Right">Right</Button>
    <Button HorizontalAlignment="Stretch"
      HorizontalContentAlignment="Left">Stretch</Button>
  </StackPanel>
  <StackPanel Orientation="Horizontal" Height="100">
    <Button VerticalAlignment="Top">Top</Button>
    <Button VerticalAlignment="Center">Center</Button>
    <Button VerticalAlignment="Bottom">Bottom</Button>
    <Button VerticalAlignment="Stretch"
      VerticalContentAlignment="Top">Stretch</Button>
  </StackPanel>
</StackPanel>
```



Verschiedene Ausrichtungseigenschaften können verwendet werden, um die Positionierung und Ausrichtung von Elementen innerhalb eines Containers zu steuern:

1. **HorizontalAlignment:** Die **HorizontalAlignment**-Eigenschaft definiert die horizontale Ausrichtung eines Elements innerhalb seines übergeordneten Containers. Mögliche Werte sind "Left" für die linke Ausrichtung, "Right" für die rechte Ausrichtung, "Center" für die zentrierte Ausrichtung oder "Stretch", um das Element über die gesamte verfügbare Breite auszudehnen.
2. **VerticalAlignment:** Die **VerticalAlignment**-Eigenschaft funktioniert ähnlich wie die **HorizontalAlignment**-Eigenschaft, jedoch für die vertikale Ausrichtung eines Elements. Möglicher Werte sind "Top" für die obere Ausrichtung, "Bottom" für die untere Ausrichtung, "Center" für die zentrierte Ausrichtung oder "Stretch" verwenden, um das Element über die gesamte verfügbare Höhe auszudehnen.

Hier ist zu beachten, dass diese Eigenschaften in bestimmten Layoutcontainern keine Auswirkungen haben. So hat in einem vertikalen **StackPanel** die Eigenschaft **VerticalAlignment** keine Funktion.

Zusätzlich kann bei Controls mit der Content-Eigenschaft diese durch die Eigenschaften **HorizontalContentAlignment** und **VerticalContentAlignment** innerhalb des Controls ausgerichtet werden. Die möglichen Werte stimmen mit den Werten der oben genannten Pendanten überein.

AttachedProperties

- ‚Von außen‘ eingefügte Eigenschaften -> sind nicht Teil der Klasse, sondern werden durch eine andere Klasse zur Verfügung gestellt
- Ermöglicht das Hinzufügen von situationspezifischen Eigenschaften, ohne diese in den einzelnen Klassen definieren zu müssen
- Verwendung hauptsächlich durch Layoutcontainer, aber auch durch Funktionsklassen, wie z.B. Validation
- Angabe in XAML innerhalb des Verwender-Tags nach dem Schema `Originalklasse.Eigenschaft`
- C#-Zugriff über statische Funktionen der Originalklasse unter Angabe des Verwenderobjekts

```
<Rectangle Canvas.Left="100"           Canvas.SetLeft(Rct_Blue, 50);  
          Canvas.Top="100"             Canvas.GetLeft(Rct_Blue);  
          x:Name="Rct_Blue"/>
```

AttachedProperties sind eine leistungsstarke Funktion in WPF, die es ermöglicht, zusätzliche Eigenschaften an ein Steuerelement anzufügen, ohne die zugrunde liegende Klasse des Steuerelements zu erweitern. Sie erweitern die Flexibilität und Erweiterbarkeit von WPF, indem sie es ermöglichen, zusätzliche Informationen an ein Steuerelement anzuhängen. Diese beeinflussen und definieren dann normalerweise Funktionen der Klasse, welche die Eigenschaften zu Verfügung stellt.

Den häufigsten Einsatz finden AttachedProperties bei der Definition von Layoutcontainer-spezifischen Einstellungen. So können beispielsweise Container-spezifische Positionsangaben für einzelne Elemente bestimmt werden. Aber auch innerhalb von Funktionsklassen wie z.B. Validation werden AttachedProperties verwendet.

Innerhalb von XAML werden diese Eigenschaften, wie ‚normale‘ Attribute in die Start-Tags der Verwenderobjekte gesetzt, haben aber hier als ‚Objektpfad‘ die Ursprungsklasse als Präfix. C#-seitig erfolgt der Zugriff über statische Getter- und Setter-Funktionen der Ursprungsklassen unter Übergabe der Referenz des Verwenderobjekts.

Layout-Container

- Canvas
- StackPanel
- WrapPanel
- Dockpanel
- Grid + UniformGrid

- ViewBox
- Border
- ScrollViewer

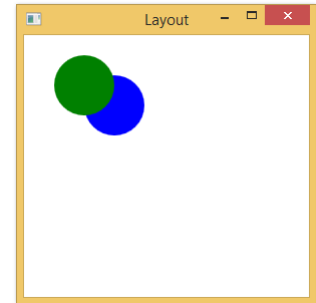
Die grundsätzliche Layoutstruktur wird durch die Auswahl und Kombination der verschiedenen Layoutcontainer erreicht. Hierbei hat jeder Container seine eigene Logik, nach der er beinhaltende Elemente anordnet oder über deren Größe und Ausrichtung entscheidet, wobei dies in den meisten Fällen angepasst werden kann.

Zusätzlich können verschiedene grafische und funktionale Zusatzelemente benutzt werden um visuelle Abgrenzungen und Hervorhebungen zu definieren oder für bestimmte Bereiche zusätzliche Funktionen bereitzustellen.

Canvas

- zuständig für die Gruppierung von Elementen anhand absoluter Positionen
- absolute Positionierung innerhalb des Canvas mittels AttachedProperties
 - `Canvas.Top/Left`
 - `Canvas.Bottom/Right`
- Reihenfolge
 - Letztes Element „ganz Oben“
 - Kann mit `Panel.ZIndex="..."` geändert werden

```
<Canvas>
  <Ellipse Fill="Green" Width="60" Height="60"
    Canvas.Left="30" Canvas.Top="30" Panel.ZIndex="1"/>
  <Ellipse Fill="Blue" Width="60" Height="60"
    Canvas.Left="60" Canvas.Top="40"/>
</Canvas>
```



Das Canvas ist ein Container, der es ermöglicht, Elemente absolut zu positionieren und zu layouten. Wichtige Aspekte:

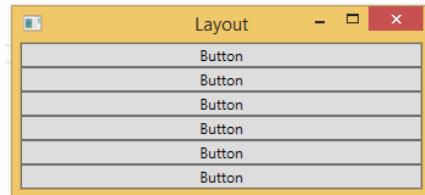
1. Absolute Positionierung: Im Canvas können Elemente an beliebigen X- und Y-Koordinaten platziert werden, wobei diese den Abstand zum linken bzw. oberen Rand des Canvas in DIP angeben. Die `Canvas.Left` und `Canvas.Top` Attached Properties können verwendet werden, um die Position eines Elements innerhalb des Canvas anzugeben.
2. Z-Achse: Das Canvas unterstützt auch die Z-Achse, was bedeutet, dass Elemente in der Stapelreihenfolge angeordnet werden können. Über `Panel.ZIndex` kann die Reihenfolge der Elemente in der Z-Achse festgelegt werden. Elemente mit höherem Z-Index werden über Elementen mit niedrigerem Z-Index gerendert.
3. Innerhalb des Canvas haben Elemente im Normalfall die Größe, die für die Darstellung ihres Inhalts benötigt wird. D.h. dass Elemente, wie das Rectangle, welche keinen Inhalt haben, in jedem Fall absolute Width- und Height-Werte benötigen

Das Canvas-Control ist besonders nützlich, wenn eine freie Positionierung von Elementen benötigt wird oder eine individuelle Anordnung von grafischen Elementen realisiert werden soll. Dies kann z.B. bei der Darstellung von Graphen oder sich bewegenden Elementen der Fall sein.

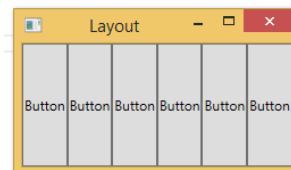
StackPanel

- Stapelt die enthaltenen Elemente
- Ausrichtung entweder Horizontal oder Vertikal über die `Orientation`-Property
- Elemente können über Containergröße hinaus laufen -> Überzählige Elemente sind nicht sichtbar

```
<StackPanel Orientation="Vertical">
    <Button>Button</Button>
    <Button>Button</Button>
    <Button>Button</Button>
</StackPanel>
```



```
<StackPanel Orientation="Horizontal">
    <Button>Button</Button>
    <Button>Button</Button>
    <Button>Button</Button>
</StackPanel>
```



Das StackPanel ist ein Container, der die Elemente in einer bestimmten Richtung stapelt. Wichtige Aspekte:

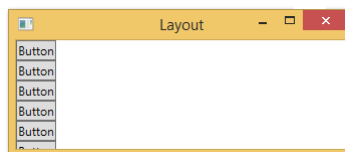
1. Ausrichtung: Das StackPanel-Control ermöglicht das Stapeln von Elementen in einer horizontalen oder vertikalen Richtung. Über Setzen der `Orientation`-Eigenschaft auf "Horizontal" oder "Vertical" kann die Stapelrichtung festgelegt werden.
2. Stapelreihenfolge: Elemente werden in der Reihenfolge ihres Hinzufügens zum StackPanel gestapelt. Das erste hinzugefügte Element befindet sich am Anfang des Stapels, das letzte hinzugefügte Element am Ende des Stapels.
3. Dynamisches Layout: Das StackPanel passt sich dynamisch an die Größe der enthaltenen Elemente an. Es wächst automatisch, um den verfügbaren Platz zu füllen, und die Elemente werden entsprechend angeordnet.
4. Überlaufbehandlung: Wenn die Größe der Elemente im StackPanel den verfügbaren Platz überschreitet, kann dies zu einem Überlauf führen.
5. Elemente innerhalb eines StackPanels sind auf der `Orientation`-Achse standardmäßig über die komplette Länge des Containers ausgedehnt und auf der anderen Achse so groß wie ihr Inhalt. Auf der `Orientation`-Achse hat die entsprechende `Alignment`-Eigenschaft keine Auswirkung.

Das StackPanel ist ideal, wenn Elemente einfach in einer linearen Anordnung gestapelt werden sollen, ohne komplexe Layouts zu erstellen. Es eignet sich gut für einfache Szenarien, in denen eine flexible Stapelung von Elementen erforderlich ist, z.B. für Listen, Schaltflächenleisten oder vertikale/horizontale Menüs.

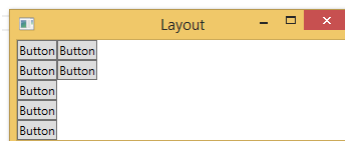
WrapPanel

- Funktioniert ähnlich wie das StackPanel
- Elemente, die nicht mehr in den Container passen, werden in eine neue Zeile/Spalte umgebrochen (=„Textfluss“)

```
<StackPanel Orientation="Vertical">  
    <Button>Button</Button>  
    <Button>Button</Button>  
    <Button>Button</Button>  
</StackPanel>
```



```
<WrapPanel Orientation="Vertical">  
    <Button>Button</Button>  
    <Button>Button</Button>  
    <Button>Button</Button>  
</WrapPanel>
```



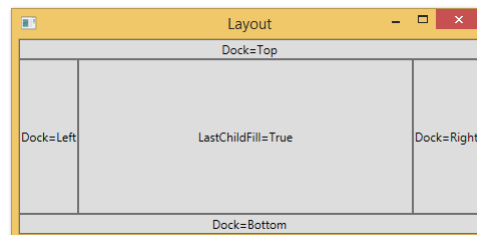
Das WrapPanel funktioniert weitestgehend wie das StackPanel, hat aber als zusätzliche Mechanik einen automatischen ‚Zeilenumbruch‘-Mechanismus. Dies bedeutet, dass wenn der benötigte Platz der beinhaltenden Elemente den zur Verfügung stehenden Platz überschreitet (auf der Orientation-Achse), wird automatisch eine weitere Zeile/Spalte eröffnet, in welche die überzähligen Elemente eingefügt werden.

Das WrapPanel- eignet sich gut für Szenarien, in denen eine flexible Anordnung von Elementen erforderlich ist, die sich automatisch an die verfügbare Fläche anpasst. Es ist besonders nützlich, wenn eine dynamische Anzahl von Elementen platziert werden sollen, die je nach Größe des Panels umgebrochen werden sollen. Beispiele für die Verwendung sind Bildergalerien, Tag-Wolken oder Listen mit variabler Länge.

DockPanel

- „Dockt“ Elemente an 5 unterschiedliche Bereiche
 - Top, Bottom, Left, Right
 - undefinierte Elemente werden in die Mitte gesetzt
 - LastChildFill-Eigenschaft definiert, ob das letzte undefinierte Element den gesamten restlichen verfügbaren Platz automatisch ausfüllt

```
<DockPanel LastChildFill="True">
  <Button DockPanel.Dock="Top">Top</Button>
  <Button DockPanel.Dock="Bottom">Bottom</Button>
  <Button DockPanel.Dock="Left">Left</Button>
  <Button DockPanel.Dock="Right">Right</Button>
  <Button>Button</Button>
</DockPanel>
```



- Die Reihenfolge der Definition (Top, Bottom, Left, Right) innerhalb von XAML beeinflusst das Layout

Das DockPanel in WPF ist ein Container, der die enthaltenen Elemente an den Rändern des Panels andockt. Wichtige Aspekte:

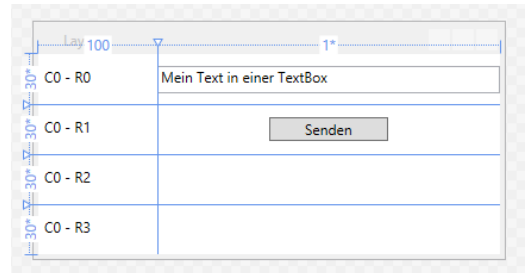
1. Anordnung an den Rändern: Das DockPanel ermöglicht das Anordnen von Elementen an den Rändern des Panels (Top, Left, Right, Bottom). Über die Angabe der DockPanel.Dock Attached Property kann festgelegt werden, an welchem Rand das Element andockt werden soll.
2. Stapelreihenfolge: Elemente werden in der Reihenfolge ihres Hinzufügens zum DockPanel angezeigt. Das erste hinzugefügte Element wird an den entsprechenden Rand gedockt und die nachfolgenden Elemente werden daran ausgerichtet.
3. Elemente, bei welchen Dock-Eigenschaft nicht gesetzt wurden werden nebeneinander in der Mitte des Containers platziert. Ist die LastChildFill-Eigenschaft auf ‚True‘ gesetzt worden, füllt das zuletzt hinzugef

Grid

- Elemente werden auf einem Gitter aus Zeilen und Spalten ausgerichtet
- Spalten und Zeilen haben eine eigene Größenangabe
- Größenangaben können relativ oder absolut sein
- UniformGrid als ‚vereinfachtes‘ Grid

```
<Grid>
  <Grid.RowDefinitions>
    <RowDefinition Height="*" />
    <RowDefinition Height="*" />
  </Grid.RowDefinitions>
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="100" />
    <ColumnDefinition Width="1*" />
  </Grid.ColumnDefinitions>

  <Label Grid.Row="0" Grid.Column="0">C0 - R0</Label>
</Grid>
```



Das Grid ist ein Container, der ein flexibles Rasterlayout für die Anordnung von Elementen bietet. Wichtige Eigenschaften:

Rasterstruktur: Das Grid besteht aus einer Rasterstruktur, die aus Zeilen und Spalten besteht. Du kannst Zeilen und Spalten definieren und Elemente in bestimmten Zellen des Rasters positionieren.

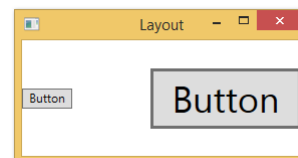
1. Zeilen- und Spaltendefinitionen: Die Anzahl der Zeilen und Spalten im Grid werden in den entsprechenden Eigenschaften mittels RowDefinition- bzw ColumnDefinition-Objekten definiert und ihre Größen festgelegt werden. Es können feste Größen (in DIP) verwendet werden oder automatische Größen angegeben werden, die sich dynamisch an den verfügbaren Platz anpassen („*“ bzw multiplizierte *, wie „2*“) oder an dem Inhalt der Zeile/Spalte orientieren („Auto“).
2. Zellpositionierung: Elemente werden in bestimmten Zellen des Grid positionieren, indem die Grid.Row- und Grid.Column-Attached Properties verwendet werden. Mit Grid.RowSpan- und Grid.ColumnSpan können Elemente über mehrere Zeilen oder Spalten ausgedehnt werden.

Alternativ kann das UniformGrid als eine vereinfachte Version des Grid verwendet werden, welches kaum Konfiguration benötigt, aber dafür auch nicht sonderlich anpassbar ist.

ViewBox

- Skaliert Elemente automatisch auf die Größe der ViewBox
- Stretch funktioniert wie bei Image-Objekt
- StretchDirection
 - UpOnly – Content wird nur hochskaliert
 - DownOnly – Content wird nur herunterskaliert
 - Both – Skalierung des Contents in beide Richtungen

```
<Button Height="20" Width="50">Button</Button>
<Viewbox Width="150" Height="150">
  <Button Height="20" Width="50">Button</Button>
</Viewbox>
```

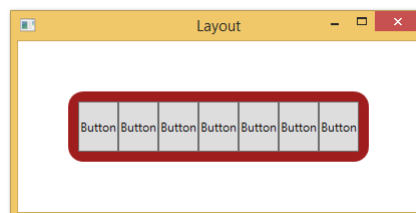


Das ViewBox-Control ermöglicht die skalierbare Darstellung von Inhalten durch automatische Anpassung an die verfügbare Größe und Wiederherstellung des Seitenverhältnisses. Es passt sich an den übergeordneten Container an und bietet Zentrierungs- und Skalierungsoptionen. ViewBox kann auch verschachtelt werden, um komplexe Layouts zu erstellen. Es ist ideal für Inhalte wie Bilder und Texte, aber auch andere Controls, die in ihrer Größe variieren und flexibel skaliert werden müssen. Hierbei verhält sich der Inhalt, wie die Bilder innerhalb eines Images-Controls.

Border

- Kann nur ein Element enthalten
 - z.B. einen Layout-Container
- Ist für die Darstellung eines Rahmens zuständig
- Einige Elemente (z.B. Button, Label) besitzen bereits einen Rahmen
- Alternative: z.B. GroupBox zu grafischen Gruppierung von Elementen

```
<Border CornerRadius="5" BorderBrush="Red"
BorderThickness="5">
  <StackPanel>
    <Button>Button</Button>
    <Button>Button</Button>
    <Button>Button</Button>
  </StackPanel>
</Border>
```



Das Border-Control ermöglicht eine grafische Umrandung eines einzelnen Elements (Control oder Container). Dicke und Farbe können frei angepasst werden, ebenso wie z.B. die Abrundung der Ecken.

Manche Elemente, wie z.B. das Label haben bereits einen Border integriert und bieten die entsprechenden Eigenschaften deshalb ebenfalls an.

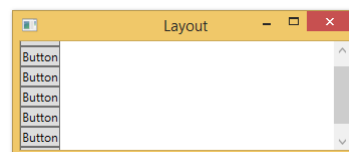
Alternativ können andere grafische Elemente zu visuellen Unterteilung verwendet werden:

1. Die GroupBox zeichnet ebenfalls einen Rahmen um das beinhaltende Element, gibt aber die Möglichkeit, einen Titel anzuzeigen. Daher ist es besonders geeignet, um funktionale Gruppen von Controls grafisch zusammenzufassen.
2. Der Separator ist ein horizontaler Strich, welcher insbesondere innerhalb von Menüs zur grafischen Abgrenzung verwendet wird.

ScrollView

- Erstellt automatisch eine Scrollbar falls dies für die Darstellung des Inhalts notwendig ist
- Kann nur ein Element beinhalten
 - Üblicherweise ein anderer Container (zB. StackPanel)
- Horizontal / Vertical – ScrollBarVisibility
 - Disabled – Deaktiviert, kann nicht scrollen
 - Hidden – unsichtbar und kann scrollen
 - Visible – immer sichtbar
 - Auto – automatische Einblendung der Scrollbar, wenn notwendig

```
<ScrollView VerticalScrollBarVisibility="Auto">
    <StackPanel>
        <Button>Btn</Button>
        <Button>Btn</Button>
        <Button>Btn</Button>
    </StackPanel>
</ScrollView>
```



Das ScrollView-Control ermöglicht das Scrollen von Inhalten, die größer sind als der sichtbare Anzeigebereich. Es zeigt automatisch horizontale und vertikale Scrollbalken an und bietet verschiedene Methoden zum Scrollen und Zoomen des Inhalts. Mit dem ScrollView können Inhalte ausgerichtet und auch verschachtelt werden, um in mehreren Richtungen zu scrollen. Es ist besonders nützlich, um lange Texte, Tabellen oder Bilder darzustellen, die nicht komplett auf dem Bildschirm angezeigt werden können.

Auch der ScrollView ist z.T. schon in anderen Controls integriert. Zum Beispiel bietet auch die TextBox entsprechende Eigenschaften für den Inhalt an.